LAB MANUAL

AI-Assisted Cyber Operations and Authorized Testing

Lab 7: Authorized Assessment of the Staging Portal · Lab 8: Redesign a Legacy Lab as Purple Team

Date	Friday, November 13, 2026
Time	1:30 to 5:30 p.m. ET, online
Facilitator	Frazier A. Smith, Ed.D., Central Piedmont Community College
You need	A browser with a terminal tab and an Open WebUI tab on the range; your Evidence Log; your Legacy Lab Inventory sheet from prework; the handout, deliverable, and rubric of the lab you will redesign, with student names removed; the raw output sample you saved in prework

Everything in these labs is synthetic, and nobody scans anything today. Northbridge Technical College, its staging network, and its people are fictional, and every hostname ends in .example. The Northbridge IT Security Office recorded the scans, HTTP responses, and package inventory inside the authorized window, and Lab 7 works from those recorded captures. The work is interpretation, scope, and evidence, not running scanners. The captures and telemetry are Confidential in the story, so they stay on the campus AI server. The only real-world material you use today is the public NVD and CISA KEV record for CVE-2021-41773.

This material is based upon work supported by the U.S. National Science Foundation under Grant No. 2300188. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the author(s) and do not necessarily reflect the views of the National Science Foundation.

Before You Start

Range Access

1. Open the terminal at `nitic-lab.labsconnect.org` and sign in as your range account (`nwXX`), the account you used in Sessions 1 to 3.
2. Open Open WebUI at `nitic-chat.labsconnect.org` in a second browser tab and sign in.
3. In the terminal, confirm the session files are present and intact:

```
ls /opt/nitic/session4/
cd /opt/nitic/session4 && sha256sum -c --quiet SHA256SUMS && echo "all files verified"
```

You should see `lab7/`, `lab8/`, `README.md`, and `SHA256SUMS`. The second command prints `all files verified` when every file matches its hash. Any line it prints before that names a file that failed. Report a failure to the facilitator in chat.

4. Open your Evidence Log. The columns are the same as in Sessions 1 to 3. Keep adding rows.

Model Names Used in This Manual

Placeholder	Range (Ollama) default	Use
`SMALL_MODEL`	`gemma3:4b`	Faster answers when the large model is slow (`--small`)
`LARGE_MODEL`	`gpt-oss:20b`	Default for `hypothesize.sh`

Both models run on the campus AI server. If the range GPU is unavailable, the facilitator hands out saved model outputs and the analyst critiques those. No Lab 7 file goes to a hosted model.

Nobody Scans Today

The Northbridge IT Security Office recorded these captures inside the authorized window from its assessment workstation. Today's lab is about interpretation, scope, and evidence, not about running scanners. The captures are authentic tool output: real `nmap -sT -sV` and `curl` runs against the staging services, recorded by the security office operator, Marcus Bell, on `sec-scan01` (10.40.13.50). Your team reads the captures, records what they show, asks a model for hypotheses, validates those hypotheses against other captures, and writes the approval request for the one consequential test. Kwame Mensah answers the request, and the facilitator releases the recorded result of an approved test. No command in this manual sends traffic to any address.

The Data Rule for This Session

The captures, the telemetry, and the package inventory are **Confidential** under the Northbridge interim AI data use policy. They may go into `hypothesize.sh` and into Open WebUI, because both run on the campus AI server. Never paste a capture, a log line, or a finding into any hosted AI service. Describe any exposed file by name, size, and timestamp only. Keep your working copies in your range home directory.

Where Things Are

Path	What it is
`lab7/authorization_mensah_2026-11-12.md`	Kwame Mensah's signed authorization for the assessment
`lab7/rules_of_engagement.md`	The rules of engagement: seven elements and the approval rule
`lab7/pilot_decision_request.md`	The provost's and CIO's request for a go or no-go decision by November 16
`lab7/staging_network_note.md`	The network team's description of the staging subnet, its sensor, and its gateway
`lab7/captures/`	Recorded captures, one file per capture, each named with its capture ID
`lab7/telemetry/`	The staging sensor's Zeek `conn.log` and `http.log`, Suricata `eve.json`, and the portal's nginx access log, with a README
`lab7/injects/`	Empty until the break
`lab7/bin/hypothesize.sh`	Sends named captures plus the rules of engagement to the campus model with the four-part hypothesis request
`lab7/bin/llm_chat.py`	Helper that `hypothesize.sh` uses to call the local model
`lab7/templates/`	Scope read-back, observation log, hypothesis record, approval request, timeline reconciliation, finding write-up, detection and remediation, and a CVE-2021-41773 reference
`lab8/purple_team_lab_canvas.md`	The Lab 8 canvas

All paths are under `/opt/nitic/session4/`. The files there are read-only.

The Captures

ID	File	What it is
C00	`C00_network_team_sweep_2026-11-11.txt`	Network team sweep, November 11
C01	`C01_name_resolution.txt`	Name resolution of the two target names
C02	`C02_nmap_sT_sV_portal-stg.txt`	`nmap -sT -sV` of the portal host
C03	`C03_nmap_sT_sV_api-stg.txt`	`nmap -sT -sV` of the API host
C04	`C04_http_portal-stg.txt`	Read-only HTTP requests to the portal
C05	`C05_http_api-stg.txt`	Read-only HTTP requests to the API
C06	`C06_staging_package_inventory.txt`	The staging package inventory for both hosts

The table lists what each file is as recorded. Deciding whether the rules of engagement let you use each file is part of the lab. One more capture, `C07_curl_backup_index_APPROVED_1449.txt`, appears in `captures/` at the break for teams whose approval request was approved.

Every capture except `C00` opens with the same header block, then the tool's output unedited. This is the header of `C03`:

```
# Northbridge IT Security Office: assessment capture
# Capture: C03
# Operator: Marcus Bell, IT Security Office, on sec-scan01 (10.40.13.50)
# Authorization: K. Mensah, 2026-11-12; window 2026-11-13 14:00 to 16:00 ET
# Tool: Nmap 7.98
# Command: nmap -sT -sV -Pn 10.40.13.11
# Started: 2026-11-13 14:04:12 EST (19:04:12 UTC)
# ----- raw output follows, unedited -----
```

`C00` has a different header, because the security office did not record it. Read that header in Part A, Step 1.

Copy the Folder

Work on your own copy of `lab7`:

```
cp -r /opt/nitic/session4/lab7 ~/lab7 && cd ~/lab7
ls captures telemetry templates
```

Every Lab 7 command below runs from `~/lab7`. If you open a new terminal tab, run `cd ~/lab7` first.

The Lab Tool

`hypothesize.sh` sends the captures you name to `LARGE_MODEL`, always with `rules_of_engagement.md` attached as the scope. Name captures by ID (`C03`) or by path, then `--`, then the question in quotes. Before the answer, the script prints the model, the time in UTC and Eastern Time, and each input file with its SHA-256 and line count. After the answer, it prints a `[stats]` line with the prompt size and run time. Copy both into your hypothesis record.

```
bin/hypothesize.sh [--small] [--show-prompt] [--thinking] FILE [FILE...] -- "question"
```

Option	What it does
`--small`	Uses `SMALL_MODEL` instead of `LARGE_MODEL`
`--show-prompt`	Prints the full prompt the model would receive, then exits without running it
`--thinking`	Also prints the model's reasoning (`gpt-oss` only)

The script accepts only files inside a `lab7` folder. It refuses `C00`, because the rules of engagement exclude that capture.

LAB 7 · 2:05 TO 4:00 · TEAMS OF THREE

Lab 7 Opening: Scope Read-Back

Time: 1:30 to 1:45, teams of three in breakout rooms for 7 minutes. **Deliverable:** A signed scope read-back, posted in the shared folder by 1:45

Do not open `captures/` until the facilitator says so. Open only the rules of engagement and the read-back template.

Step 1: Read the Scope Aloud

One member reads the rules of engagement aloud while the other two follow:

```
cd ~/lab7
less rules_of_engagement.md
```

Press `q` to leave `less`.

Step 2: Write the Read-Back

Copy the template and fill it:

```
cp templates/scope_readback.md ~/lab7/my_readback.md && nano ~/lab7/my_readback.md
```

In `nano`, save with Ctrl+O and Enter, and exit with Ctrl+X. The read-back has three parts:

1. One sentence per element in your team's own words: Target, Time window, Allowed techniques, Prohibited actions, Data rules, Approval required, Stop conditions, and Reporting path.
2. The six proposed actions in the template. Classify each as in scope, out of scope, or requires approval, and name the element that decides it.
3. The names of all three members and the time you signed.

Post the read-back in the shared folder. The read-back is the first scored act of Lab 7.

LAB 7 · 2:05 TO 4:00 · TEAMS OF THREE

Lab 7 Part A: Inventory, Hypotheses, Validation, Approval

Time: 2:05 to 2:55, teams of three. The approval request is due at 2:45. **Deliverables:** Observation log, hypothesis record, approval request, and the first timeline entries

Scenario

Northbridge plans to pilot an AI-enabled student-support portal on December 1. On November 11, the portal developers forwarded odd entries from the staging access log to the IT Security Office. On November 12, Provost Dr. Alicia Harmon and Chief Information Officer Dr. Samuel Okonjo asked for a go or no-go recommendation by Monday, November 16. The same day, Information Security Officer Kwame Mensah signed a written authorization for a limited assessment of two staging hosts, today from 2:00 to 4:00 p.m. ET. Your team is his assessment team.

The decision: **Should the AI-enabled student-support portal start its December 1 pilot as planned, start it only after named conditions are met, or wait?** Your recommendation must separate what the captures show from what a banner, a scanner, an alert, or a model claims.

Team Roles

Roles rotate at the break. All three roles produce workbook rows.

Role	Job
Operator of record	Runs every command on the range and keeps the observation log
Analyst	Runs `hypothesize.sh` and keeps the hypothesis record
Scope officer	Checks every proposed step against the rules of engagement, owns the approval request, and calls any stop condition

Stop Conditions

If a result or a proposed step points outside the two targets, stop that line of work. Preserve your record, then post the stop to "Kwame Mensah" in the session chat:

```
history > ~/lab7/stop_record.txt
```

Part A, Step 1: Inventory (8 minutes, 2:05 to 2:13)

Read the network team's note, then the name resolution and the two service scans:

```
cd ~/lab7
less staging_network_note.md
less captures/C01_name_resolution.txt captures/C02_nmap_sT_sV_portal-stg.txt captures/C03_nmap_sT_sV_api-stg.txt
grep -n "open" captures/C0[23]_*
```

In `less`, type `:n` to move to the next file. List each in-scope host, its port, and the service the tool reported.

Then read the header of the network team's sweep:

```
head -12 captures/C00_network_team_sweep_2026-11-11.txt
```

Read the header and decide whether the rules of engagement let you use this file. Record the decision, and the element that decides it, in your observation log.

Part A, Step 2: Raw Output Record (7 minutes, 2:13 to 2:20)

Record every capture before anyone asks a model about it. Hash the captures, and read the header lines that name the tool, the command, the target, and the time:

```
sha256sum captures/*.txt
grep '^# ' captures/C03_*
cp templates/observation_log.md ~/lab7/my_observation_log.md && nano ~/lab7/my_observation_log.md
```

Run the `grep` line for each capture ID in turn. Write one observation-log row per capture. The columns are:

Column	What goes there
Row	A running number
Capture or file	The file name
SHA-256	The hash from `sha256sum`
Tool and version	As the header and the output state it
Command	The exact command in the header
Target as authorized	The host, address, and port as element 1 of the rules of engagement writes them
Start time and zone	The start time with its time zone
Observed	Only what the tool printed
Excluded? and why	Yes or no, and the deciding element

Write only what the tool printed in the Observed column. An inference goes in the hypothesis record.

Part A, Step 3: Ranked Hypotheses (10 minutes, 2:20 to 2:30)

The analyst asks the model about each scan and saves every answer:

```
bin/hypothesize.sh C03 -- "What service is listening on 10.40.13.11:8080, and is it vulnerable?" | tee
~/lab7/hyp_C03.txt
bin/hypothesize.sh C02 -- "What service is listening on 10.40.13.10:80, and is anything exposed?" | tee
~/lab7/hyp_C02.txt
```

If the large model is slow, run the same question on the small model:

```
bin/hypothesize.sh --small C03 -- "What service is listening on 10.40.13.11:8080, and is it vulnerable?" |
tee ~/lab7/hyp_C03_small.txt
```

To see exactly what the model receives, print the prompt without running it:

```
bin/hypothesize.sh --show-prompt C03 -- "test"
```

Copy the hypothesis record template and fill one row per hypothesis:

```
cp templates/hypothesis_record.md ~/lab7/my_hypothesis_record.md && nano ~/lab7/my_hypothesis_record.md
```

The columns are Hypothesis, Source (model and time, or team), Evidence to confirm, Evidence to rule out, Proposed next step, Scope verdict for the step, Result, and Correction. The scope officer marks every proposed next step in scope, out of scope, or requires approval, and names the deciding element. A model's recommendation is never approval.

Part A, Step 4: Validate Service Identity (10 minutes, 2:30 to 2:40)

Before you accept any service, version, or vulnerability claim, collect at least two pieces of evidence that the service banner did not supply. Read the HTTP captures and the package inventory:

```
less captures/C05_http_api-stg.txt captures/C04_http_portal-stg.txt
grep -n -A100 "== api-stg ==" captures/C06_staging_package_inventory.txt | less
grep -n -i "apache\|nginx" captures/C06_staging_package_inventory.txt
```

Compare what each response body says, which headers each response carries, the order of those headers, and what an error page looks like on each host. Compare the packages installed on each host with the product each banner names.

Then read the staging sensor's HTTP log for the API port. Zeek logs are tab-separated, and the `#fields` line names the columns. Print the column numbers first:

```
grep '^#fields' telemetry/zeek_http.log | tr '\t' '\n' | tail -n +2 | cat -n
```

The numbers printed match the `awk` field numbers. Column 6 is `id.resp_p`, the destination port. Column 8 is `method`, 10 is `uri`, 13 is `user_agent`, 16 is `response_body_len`, 17 is `status_code`, and 24 is `proxied`. Show every request to port 8080, and then only the columns you need:

```
grep -v '^#' telemetry/zeek_http.log | awk -F'\t' '$6=="8080"' | less -S
grep -v '^#' telemetry/zeek_http.log | awk -F'\t' '$6=="8080" {print $1, $3, $8, $10, $17, $16}'
```

In `less -S`, use the arrow keys to scroll sideways. Ask the model again with all three captures in context, and save the answer:

```
bin/hypothesize.sh C03 C05 C06 -- "Given all three, what service is on 10.40.13.11:8080?" | tee
~/lab7/hyp_C03_C05_C06.txt
```

Record the verdict for each host, the evidence for it with the capture and line, and your confidence. Record any correction to an earlier hypothesis in the Correction column. Do not delete the earlier row.

Part A, Step 5: Approval Request (5 minutes, due 2:45)

Any request beyond the paths in element 3 of the rules of engagement needs written approval from Kwame Mensah before it runs. The scope officer writes the request:

```
cp templates/approval_request.md ~/lab7/my_approval_request.md && nano ~/lab7/my_approval_request.md
```

The request has four parts:

1. The finding so far, with a pointer to the raw output (capture ID and line).
2. The proposed test and its expected effect on the target.
3. A less invasive alternative, and why it is or is not sufficient.
4. How the target returns to its prior state.

Fill "Requested by, time." Post the request in the session chat to "Kwame Mensah" and wait for the reply. Record the reply and its time under "Approver response, time." A model's recommendation is never approval. A returned request names the missing part, so fix that part and post it again.

Part A, Step 6: Start the Timeline (10 minutes, 2:45 to 2:55)

List the November 10 to 11 events from the telemetry in UTC, with the source address, the sensor, and the file for each. Do not convert time zones yet. Part B needs the raw times.

Read the portal's access log without the health checks, then the Suricata alerts:

```
grep -v '^#' telemetry/portal-stg_access.log | grep -v -i "healthz" | less
grep '"event_type":"alert"' telemetry/suricata_eve.json | python3 -m json.tool --json-lines | less
```

Zeek records time as `ts`, seconds since January 1, 1970 UTC. Convert a `ts` value to a date with `date`. The example below converts the `ts` of the first row in `zeek_conn.log`, a health check on the portal:

```
date -u -d @1794315517.630327
```

It prints `Tue Nov 10 12:58:37 UTC 2026`.

Copy the timeline template and enter one row per event:

```
cp templates/timeline_reconciliation.md ~/lab7/my_timeline.md && nano ~/lab7/my_timeline.md
```

The columns are Event, Source file and sensor, Time as recorded, Zone, Corrected UTC, Source address as recorded, Client after reconciliation, and Notes. Leave Corrected UTC and Client after reconciliation blank until Part B.

LAB 7 · 2:05 TO 4:00 · TEAMS OF THREE

Lab 7 Part B: Releases, Purple-Team Extension, and Briefing

Time: 2:55 to 4:25. The break and releases run 2:55 to 3:05, Part B runs 3:05 to 4:00, and briefings run 4:00 to 4:25. **Deliverables:** Revised hypotheses, timeline reconciliation, finding write-up, detection and remediation, Evidence Log rows, a three-minute briefing, and the written finding to Kwame Mensah by 4:25

Break and Releases (2:55 to 3:05)

During the break the facilitator releases two kinds of files on the range:

- Teams whose approval request was approved find `C07_curl_backup_index_APPROVED_1449.txt` in `/opt/nitic/session4/lab7/captures/`. A team without an approved request posts a complete request first.
- Every team finds two items in `/opt/nitic/session4/lab7/injects/`: `I1_vulnwatch_export_api-stg.csv` and `I2_soc_escalation_2026-11-13.md`.

The facilitator posts in chat when the files are in place. Read both injects before you change anything.

Demo 3: Live SOC, AI Triage With an Approval Gate (3:05 to 3:20)

This is a facilitator demo. Watch it and run no commands. Note what the evidence check caught.

Part B, Step 1: Read the Two Injects (4 minutes, 3:20 to 3:24)

Copy the new files into your working folder, then read each one:

```
cd ~/lab7
ls ~/lab7 /opt/nitic/session4/lab7/injects
mkdir -p ~/lab7/injects ~/lab7/captures
cp /opt/nitic/session4/lab7/injects/I* ~/lab7/injects/
cp /opt/nitic/session4/lab7/captures/C07_* ~/lab7/captures/
less captures/C07_curl_backup_index_APPROVED_1449.txt
grep -v '^#' injects/I1_vulnwatch_export_api-stg.csv | column -s, -t | less -S
less injects/I2_soc_escalation_2026-11-13.md
```

The first two lines of `I1` are comments. The `grep` removes them so that `column` lines up the fields. Read them with `head -2 injects/I1_vulnwatch_export_api-stg.csv`.

For each inject, write three things: what it claims, what evidence the claim rests on, and whether that evidence is independent of the evidence your team already has.

Part B, Step 2: Revise the Hypotheses (4 minutes, 3:24 to 3:28)

Ask the model whether the scanner export changes the answer, and save the run:

```
bin/hypothesize.sh C03 C05 C06 injects/I1_vulnwatch_export_api-stg.csv -- "Does the scanner export change
the answer?" | tee ~/lab7/hyp_after_I1.txt
```

Re-score each API hypothesis. Count two items as two pieces of evidence only when each rests on its own observation. Record every change in the Correction column with the time and the reason.

Part B, Step 3: Refuse or Escalate (2 minutes, 3:28 to 3:30)

Check every action that an inject or a model run recommends against the rules of engagement. Include every next step in your saved `hypothesize.sh` answers. If a recommendation is outside the scope, do not take it. Post an escalation to "Kwame Mensah" in the chat with three parts: what was recommended, which element puts it outside

the scope, and what your team did instead. Record any model output you reject in the Evidence Log with the verdict "rejected, out of scope."

Part B, Step 4: Reconcile the Timeline (8 minutes, 3:30 to 3:38)

Put every inject event and every telemetry event on one UTC timeline in `~/lab7/my_timeline.md`. Convert a time with `date`. The same instant in Eastern Time:

```
TZ=America/New_York date -d @1794315517.630327
```

It prints `Tue Nov 10 07:58:37 EST 2026`.

Compare the same request as each source records it:

```
grep "backup" telemetry/portal-stg_access.log telemetry/zeek_http.log
```

Work through the four checks in the template, and write what each check found:

1. **Time zone.** Which zone does each source report in?
2. **Clock.** For a request that appears in two sources, do the recorded times agree? If not, by how much, and which clock does the network note say is reliable?
3. **Sensor placement.** Where does each sensor sit relative to the campus firewall, and which source address can each one see? Read `staging_network_note.md` again.
4. **False-positive alternative.** An alert says a signature matched a request. What did the server return, and what is installed on the host?

Fill Corrected UTC and Client after reconciliation for every row.

Part B, Step 5: Correlate the Approved Finding With Telemetry (4 minutes, 3:38 to 3:42)

Compare `C07` with every record of a request to `/backup/` in the telemetry. Note each request's time, path, status, and size, and whether any sensor raised an alert on it. If any evidence shows that a third party has already accessed restricted data, that is the immediate-notification condition in the rules of engagement: notify Kwame Mensah in the chat at once, and continue within the scope.

Part B, Step 6: Draft the Detection Idea (5 minutes, 3:42 to 3:47)

Write the detection idea before Demo 4, so the demo tests your idea instead of replacing it:

```
cp templates/detection_remediation.md ~/lab7/my_detection_remediation.md && nano
~/lab7/my_detection_remediation.md
```

Name the data source, the condition, and the expected false positives.

Demo 4: From Finding to Detection (3:47 to 3:55)

This is a facilitator demo. Write your detection idea before it starts. Note which of your negative cases the rule missed.

Part B, Step 7: Translate the Finding (5 minutes, 3:55 to 4:00)

Revise the detection idea against what Demo 4 showed. Then fill the finding write-up:

```
cp templates/finding_writeup.md ~/lab7/my_finding.md && nano ~/lab7/my_finding.md
```

The write-up has a title, the host and service, the evidence with capture pointers, the telemetry correlation, the severity and why, a confidence statement, and the recommendation for December 1. In `my_detection_remediation.md`, complete the remediation (owner and change) and the validation step. The validation step repeats the original observation and states the expected new result. Add Evidence Log rows for every model run. Post the written finding to Kwame Mensah in the shared folder by 4:25.

Briefing Format (3 Minutes per Team, 4:00)

One member briefs the whole room. Use this order and stop at three minutes:

1. **Scope (30 seconds).** What the team was authorized to do, and one thing it refused or escalated, with the deciding element.
2. **Evidence (60 seconds).** The one finding with its capture pointer and the telemetry correlation. The verdict on the API banner with the two strongest pieces of evidence.
3. **Remediation (60 seconds).** The detection idea, the remediation and its owner, the validation step, and the confidence.
4. **Recommendation (30 seconds).** Go, conditional go with named conditions, or no-go for December 1.

Evidence Log Rows

Add one row for every `hypothesize.sh` run.

Row	Prompt	Highest data tier	Validation method	Verdict
Each `hypothesize.sh` run	The question and the files in context, as the script printed them	Confidential	The captures and telemetry that confirmed or ruled out each hypothesis	Accept, revise, or reject, with the correction

LAB 8 · 4:25 TO 5:10 · INDIVIDUAL AND PAIRS

Lab 8: Redesign a Legacy Lab as Purple Team

Time: 4:25 to 5:10. Individual work for 35 minutes, then pairs grouped by platform for a 10-minute safety review.
Deliverables: A purple-team lab canvas for your own course, saved to the shared folder; two written safety-review answers on a partner's canvas

Scenario

Your department wants students to do what your team did in Lab 7: work inside a written authorization, record raw output before any model interprets it, validate a model's claim with independent evidence, ask for approval before a consequential test, and turn the finding into detection and remediation. You will redesign the scanning or penetration-testing lab you brought.

Step 1: Copy the Canvas and Read the Example (5 minutes, 4:25 to 4:30)

```
cp /opt/nitic/session4/lab8/purple_team_lab_canvas.md ~/lab8_canvas.md
less /opt/nitic/session4/lab8/example_canvas_network_discovery.md
```

The example appears at 4:25. It fills the canvas for the prework's sample Network Discovery Lab and uses the Lab 7 answers as patterns.

Step 2: Fill the Canvas (30 minutes, 4:30 to 5:00)

```
nano ~/lab8_canvas.md
```

Start from your Legacy Lab Inventory: copy each inventory field into the matching canvas field, then change it. The canvas has twelve fields: learner and prerequisite; observable performance; scenario decision; authorized assets and stop conditions; AI role and permissions; expected failure or misconception; independent validation; student evidence; instructor truth and hints; reset and fallback; accessibility path; and workforce alignment.

The redesign must include all seven requirements:

1. **Authorization.** A written rules-of-engagement document with all seven elements and the approval rule, and a scope read-back as the first scored act.
2. **Known expected findings.** The instructor's truth for every target, including at least one planted misconception (a misleading banner, a stale scan, a decoy host, or an out-of-scope neighbor) and how a student would detect it.
3. **AI-use boundaries.** What students may ask a model, which data tier may go to which model, where the approval gate sits, and the rule that a model's suggestion is never approval. If the lab uses an agent, the gate is code that runs before execution.
4. **Validation checkpoints.** At least two points where a student confirms a model or tool claim with independent evidence before continuing.
5. **Defensive extension.** The detection idea, remediation, and validation step a student writes for the main finding, and the telemetry the student needs to write them.
6. **Reset.** How the lab returns to a known state between students and sections.
7. **Instructor troubleshooting notes.** The three most likely failures and the move for each, including a student who exceeds the scope: stop, preserve logs, debrief.

Write requirements 2 and 4 first. They are the parts that stop a student from completing the lab by asking a model.

Step 3: Safety Review (10 minutes, 5:00 to 5:10)

Swap canvases with a partner who uses the same platform. Answer two questions in writing under "Safety review" at the bottom of your partner's canvas, with your name:

1. Could a student following this canvas touch anything outside the authorized assets? Name the step, or state why none exists.
2. Could a student complete the validation checkpoints by asking a model, without looking at the evidence? Name the checkpoint, or state why none exists.

Revise your own canvas from your partner's answers, then save it to the shared folder as `S4_Lab8_<lastname>.md`.

RUBRIC

Scoring

Lab 7 is scored on 20 points:

Criterion	Points	Full credit
Scope and authorization	4	Signed read-back restates all seven elements and the approval rule; the six actions classified with deciding elements; every capture or recommendation outside the scope excluded, refused, and reported through the reporting path
Raw-output record	3	Every capture logged with tool, command, target as authorized, time with zone, and hash before any model run; observed and inferred kept in separate columns
Hypotheses and validation	5	Model hypotheses recorded with model and time; every service claim tested with at least two independent pieces of evidence; evidence that rests on the same observation counted once; corrections recorded
Approval and the least invasive finding	3	Four-part request complete before the test; the least invasive test requested; any exposed file described by name, size, and timestamp only
Purple-team translation and briefing	5	Timeline reconciled on time zone, clock, sensor placement, and false positive; detection idea with data source, condition, and false positives; remediation with owner; validation repeats the original observation; confidence stated; recommendation follows from the evidence

Lab 8 is scored on 10 points:

Criterion	Points	Full credit
Authorization and stop conditions	2	All seven elements and the approval rule; stop conditions observable by a student
Expected findings and misconception	2	Instructor truth for every target; one planted misconception with a detection path
AI-use boundaries and gate	2	Data tiers per model; gate placement stated; model suggestion is not approval
Validation and defensive extension	2	Two checkpoints a model cannot satisfy alone; detection, remediation, and validation for the main finding
Reset, troubleshooting, and accessibility	2	Reset named; three failures with moves; an accessibility path that reaches the same performance

Save the Lab 7 read-back by 1:45, the approval request in chat by 2:45, the written finding by 4:25, and the Lab 8 canvas by 5:10.